Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Доповідь з дисципліни
«Об'єктно-орієнтоване програмування»

на тему:
«Розробка інтерфейсу користувача»

(третя лабороторна робота)

Виконав:

Студент групи ІМ-51
Круковський Володимир Шон Ігорович
номер у списку групи: 20

Київ – 2026

Мета роботи:

Ознайомлення з принципами створення панелі інструментів (Toolbar), підказок (Tooltips), динамічного виділення пам'яті для масиву об'єктів геометричних фігур, а також реалізація індивідуального варіанта завдання з використанням поліморфізму в Win32 API.

Завдання згідно варіанту:

1. Реалізувати динамічний масив фігур із розміром $N = 121$ (Shape** pcshape).
2. Додати панель інструментів (Toolbar), яка дублює пункти меню вибору об'єктів («Точка», «Лінія», «Прямокутник», «Еліпс»).
3. Реалізувати підказки (Tooltips) для кнопок панелі інструментів за допомогою повідомлення TTN_GETDISPINFOW.
4. Змінити логіку малювання фігур згідно з індивідуальним варіантом:

    - **Прямокутник (RectShape)**: малюється з центру, має кольоровий контур (синій) і без заповнення.
    - **Еліпс (EllipseShape)**: малюється за двома протилежними кутами, має кольоровий контур (синій) та кольорове заповнення.

5. Гумовий слід (rubberband) під час перетягування мишею реалізувати за допомогою штрихової лінії (PS_DASH).

Головний файл: Lab3.cpp:

```
#include <windows.h>
#include <commctrl.h>
#include "Shape.h"

#pragma comment(lib, "comctl32.lib")

#define ID_FILE_CLEAR   101
#define ID_FILE_EXIT    102

#define ID_OBJ_POINT    201
#define ID_OBJ_LINE     202
#define ID_OBJ_RECT     203
#define ID_OBJ_ELLIPSE  204

HINSTANCE hInst;
HWND g_hWnd = NULL;
HWND g_hWndToolbar = NULL;

const int MAX_SHAPES = 121; // Варіант 21: розмір масиву 121
Shape** pcshape = nullptr; // Динамічний масив вказівників
int shapeCount = 0;

int currentShapeType = ID_OBJ_POINT;
```

```
bool isDrawing = false;
int startX = 0, startY = 0;
int currentX = 0, currentY = 0;

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_
LPWSTR lpCmdLine, _In_ int nCmdShow) {
    hInst = hInstance;

    INITCOMMONCONTROLSEX icx;
    icx.dwSize = sizeof(INITCOMMONCONTROLSEX);
    icx.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icx);

    WNDCLASSEXW wcex = { sizeof(WNDCLASSEX) };
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.hInstance = hInstance;
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszClassName = L"Lab3Class";
    RegisterClassExW(&wcex);

    HMENU hMenu = CreateMenu();
    HMENU hFileMenu = CreateMenu();
    HMENU hObjMenu = CreateMenu();
    HMENU hHelpMenu = CreateMenu();

    AppendMenuW(hFileMenu, MF_STRING, ID_FILE_CLEAR, L"Clear");
    AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);
    AppendMenuW(hFileMenu, MF_STRING, ID_FILE_EXIT, L"Exit");

    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_POINT, L"Point");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_LINE, L"Line");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_RECT, L"Rectangle");
    AppendMenuW(hObjMenu, MF_STRING, ID_OBJ_ELLIPSE, L"Ellipse");

    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"File");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hObjMenu, L"Objects");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Help");

    g_hWnd = CreateWindowW(L"Lab3Class", L"Lab 3 - Variant 21",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0, 800, 600, nullptr, hMenu, hInstance,
nullptr);

    if (!g_hWnd) return FALSE;

    // Створення панелі інструментів (Toolbar) з підтримкою підказок (tooltips)
    g_hWndToolbar = CreateWindowExW(0, TOOLBARCLASSNAMEW, NULL,
        WS_CHILD | WS_VISIBLE | TBSTYLE_TOOLTIPS, 0, 0, 0, 0,
        g_hWnd, (HMENU)1, hInstance, NULL);

    SendMessageW(g_hWndToolbar, TB_BUTTONSTRUCTSIZE, (WPARAM)sizeof(TBBUTTON), 0);

    TBBUTTON tbb[4] = {
        { 0, ID_OBJ_POINT, TBSTATE_ENABLED | TBSTATE_CHECKED, BTNS_BUTTON, {0}, 0,
(INT_PTR)L"Point" },
        { 1, ID_OBJ_LINE, TBSTATE_ENABLED, BTNS_BUTTON, {0}, 0, (INT_PTR)L"Line" },
        { 2, ID_OBJ_RECT, TBSTATE_ENABLED, BTNS_BUTTON, {0}, 0, (INT_PTR)L"Rectangle" },
        { 3, ID_OBJ_ELLIPSE, TBSTATE_ENABLED, BTNS_BUTTON, {0}, 0, (INT_PTR)L"Ellipse" }
```

```
    };

    SendMessageW(g_hWndToolbar, TB_ADDBUTTONS, (WPARAM)4, (LPARAM)&tbb);
    SendMessageW(g_hWndToolbar, TB_AUTOSIZE, 0, 0); // Автоматичне підлаштування розміру
тулбара

    // Виділення пам'яті під динамічний масив
    pcshape = new Shape * [MAX_SHAPES];
    for (int i = 0; i < MAX_SHAPES; i++) {
        pcshape[i] = nullptr;
    }

    ShowWindow(g_hWnd, nCmdShow);
    UpdateWindow(g_hWnd);

    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    // Звільнення динамічної пам'яті
    for (int i = 0; i < shapeCount; i++) {
        delete pcshape[i];
    }
    delete[] pcshape;

    return (int)msg.wParam;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_INITMENUPOPUP: {
        HMENU hMenu = (HMENU)wParam;
        CheckMenuItem(hMenu, ID_OBJ_POINT, MF_BYCOMMAND | (currentShapeType ==
ID_OBJ_POINT ? MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_LINE, MF_BYCOMMAND | (currentShapeType ==
ID_OBJ_LINE ? MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_RECT, MF_BYCOMMAND | (currentShapeType ==
ID_OBJ_RECT ? MF_CHECKED : MF_UNCHECKED));
        CheckMenuItem(hMenu, ID_OBJ_ELLIPSE, MF_BYCOMMAND | (currentShapeType ==
ID_OBJ_ELLIPSE ? MF_CHECKED : MF_UNCHECKED));
        break;
    }
    case WM_COMMAND: {
        int id = LOWORD(wParam);
        if (id == ID_FILE_CLEAR) {
            for (int i = 0; i < shapeCount; i++) {
                delete pcshape[i];
                pcshape[i] = nullptr;
            }
            shapeCount = 0;
            InvalidateRect(hWnd, NULL, TRUE);
        }
        else if (id == ID_FILE_EXIT) {
            DestroyWindow(hWnd);
        }
        else if (id >= ID_OBJ_POINT && id <= ID_OBJ_ELLIPSE) {
            currentShapeType = id;
            // Синхронізація вибору між меню та кнопками Toolbar
```

```
            SendMessageW(g_hWndToolbar, TB_CHECKBUTTON, ID_OBJ_POINT,
(LPARAM)MAKELONG(id == ID_OBJ_POINT ? TRUE : FALSE, 0));
            SendMessageW(g_hWndToolbar, TB_CHECKBUTTON, ID_OBJ_LINE, (LPARAM)MAKELONG(id
== ID_OBJ_LINE ? TRUE : FALSE, 0));
            SendMessageW(g_hWndToolbar, TB_CHECKBUTTON, ID_OBJ_RECT, (LPARAM)MAKELONG(id
== ID_OBJ_RECT ? TRUE : FALSE, 0));
            SendMessageW(g_hWndToolbar, TB_CHECKBUTTON, ID_OBJ_ELLIPSE,
(LPARAM)MAKELONG(id == ID_OBJ_ELLIPSE ? TRUE : FALSE, 0));
        }
        break;
    }
    case WM_NOTIFY: {
        // Обробка підказок (tooltips) для тулбара
        LPNMHDR pnmh = (LPNMHDR)lParam;
        if (pnmh->code == TTN_GETDISPINFOW) {
            LPNMTTDISPINFOW pdi = (LPNMTTDISPINFOW)lParam;
            if (pdi->hdr.idFrom == ID_OBJ_POINT) wcscpy_s(pdi->szText, L"Create Point");
            else if (pdi->hdr.idFrom == ID_OBJ_LINE) wcscpy_s(pdi->szText, L"Create
Line");
            else if (pdi->hdr.idFrom == ID_OBJ_RECT) wcscpy_s(pdi->szText, L"Create
Rectangle");
            else if (pdi->hdr.idFrom == ID_OBJ_ELLIPSE) wcscpy_s(pdi->szText, L"Create
Ellipse");
        }
        break;
    }
    case WM_LBUTTONDOWN: {
        isDrawing = true;
        startX = LOWORD(lParam);
        startY = HIWORD(lParam);
        currentX = startX;
        currentY = startY;
        SetCapture(hWnd);
        break;
    }
    case WM_MOUSEMOVE: {
        if (isDrawing) {
            HDC hdc = GetDC(hWnd);
            SetROP2(hdc, R2_NOTXORPEN);

            // Варіант 21: гумовий слід штриховою лінією (PS_DASH)
            HPEN hPenOld = (HPEN)SelectObject(hdc, CreatePen(PS_DASH, 1, RGB(0, 0, 0)));

            if (currentShapeType == ID_OBJ_LINE) {
                MoveToEx(hdc, startX, startY, NULL); LineTo(hdc, currentX, currentY);
                MoveToEx(hdc, startX, startY, NULL); LineTo(hdc, LOWORD(lParam),
HIWORD(lParam));
            }
            else if (currentShapeType == ID_OBJ_RECT) {
                int w1 = abs(currentX - startX);
                int h1 = abs(currentY - startY);
                Rectangle(hdc, startX - w1, startY - h1, startX + w1, startY + h1);
                int w2 = abs(LOWORD(lParam) - startX);
                int h2 = abs(HIWORD(lParam) - startY);
                Rectangle(hdc, startX - w2, startY - h2, startX + w2, startY + h2);
            }
            else if (currentShapeType == ID_OBJ_ELLIPSE) {
                Ellipse(hdc, startX, startY, currentX, currentY);
                Ellipse(hdc, startX, startY, LOWORD(lParam), HIWORD(lParam));
            }
```

```
            currentX = LOWORD(lParam);
            currentY = HIWORD(lParam);

            DeleteObject(SelectObject(hdc, hPenOld));
            ReleaseDC(hWnd, hdc);
        }
        break;
    }
    case WM_LBUTTONUP: {
        if (isDrawing) {
            isDrawing = false;
            ReleaseCapture();

            int endX = LOWORD(lParam);
            int endY = HIWORD(lParam);

            if (shapeCount < MAX_SHAPES) {
                if (currentShapeType == ID_OBJ_POINT) {
                    pcshape[shapeCount++] = new PointShape(startX, startY, RGB(0, 0,
0));
                }
                else if (currentShapeType == ID_OBJ_LINE) {
                    pcshape[shapeCount++] = new LineShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
                else if (currentShapeType == ID_OBJ_RECT) {
                    pcshape[shapeCount++] = new RectShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
                else if (currentShapeType == ID_OBJ_ELLIPSE) {
                    pcshape[shapeCount++] = new EllipseShape(startX, startY, endX, endY,
RGB(0, 0, 0));
                }
            }
            InvalidateRect(hWnd, NULL, TRUE);
        }
        break;
    }
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);

        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i] != nullptr) {
                pcshape[i]->Draw(hdc);
            }
        }

        EndPaint(hWnd, &ps);
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

Shape.h:

```
#pragma once
#include <windows.h>

class Shape {
protected:
    int startX, startY;
    int endX, endY;
    COLORREF color;
public:
    Shape(int x1, int y1, int x2, int y2, COLORREF col)
        : startX(x1), startY(y1), endX(x2), endY(y2), color(col) {
    }
    virtual ~Shape() {}
    virtual void Draw(HDC hdc) = 0; // Чисто віртуальна функція (поліморфізм)
};

// point
class PointShape : public Shape {
public:
    PointShape(int x1, int y1, COLORREF col) : Shape(x1, y1, x1, y1, col) {}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, color);
        HBRUSH hBrush = CreateSolidBrush(color);
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Ellipse(hdc, startX - 2, startY - 2, startX + 2, startY + 2);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        DeleteObject(hBrush);
    }
};

// line
class LineShape : public Shape {
public:
    LineShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2, col)
{}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, color);
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        MoveToEx(hdc, startX, startY, NULL);
        LineTo(hdc, endX, endY);
        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);
    }
};

// rectangle (з центру, синій контур, без заповнення)
class RectShape : public Shape {
public:
    RectShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2, col)
{}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 255));
        HBRUSH hBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
```

```
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        int w = abs(endX - startX);
        int h = abs(endY - startY);
        Rectangle(hdc, startX - w, startY - h, startX + w, startY + h);

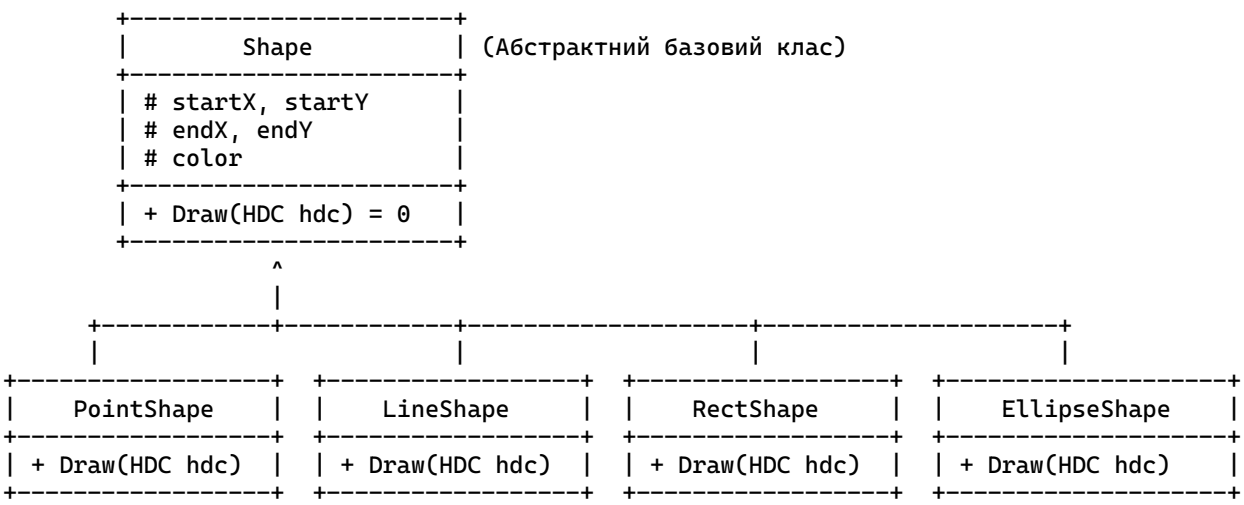
        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
    }
};

// ellipse (за двома кутами, синій контур, з заповненням)
class EllipseShape : public Shape {
public:
    EllipseShape(int x1, int y1, int x2, int y2, COLORREF col) : Shape(x1, y1, x2, y2,
col) {}
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 255));
        HBRUSH hBrush = CreateSolidBrush(RGB(200, 220, 255));
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

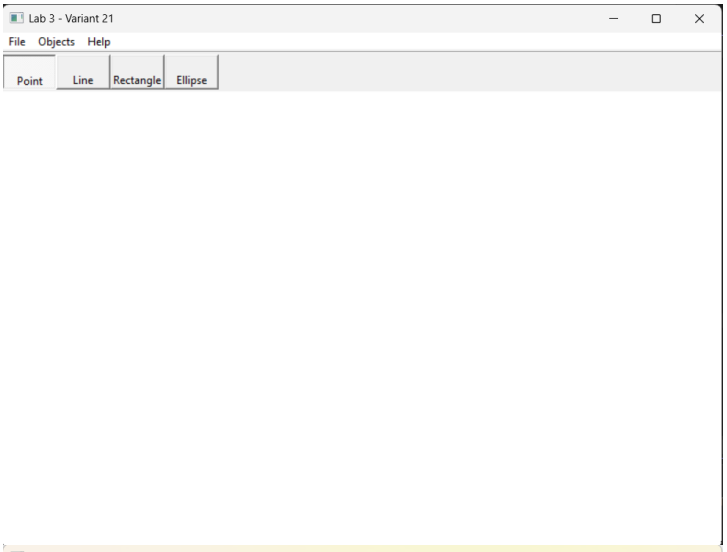
        Ellipse(hdc, startX, startY, endX, endY);

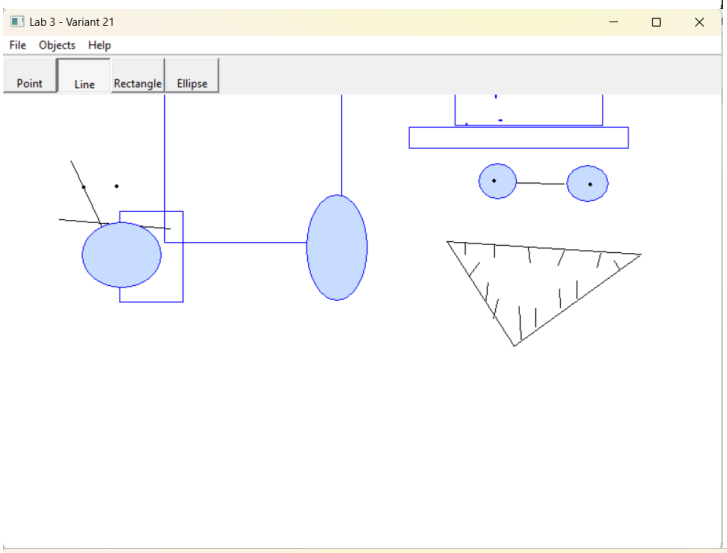
        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        DeleteObject(hBrush);
    }
};
```

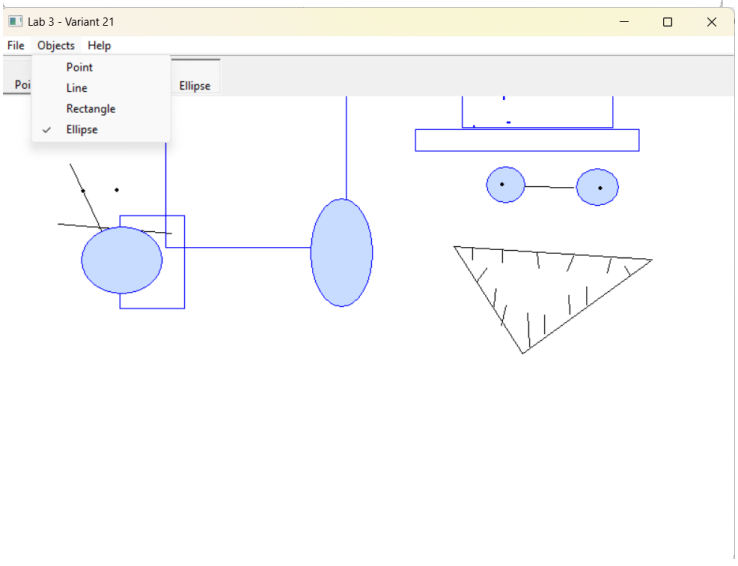
Ієрархія класів:



Скріншоти:


Lab 3 - Variant 21
File Objects Help
Point Line Rectangle Ellipse


Lab 3 - Variant 21
File Objects Help
Point Line Rectangle Ellipse


Lab 3 - Variant 21
File Objects Help
Point
Line
Rectangle
Ellipse
Ellipse

Lab 3 - Variant 21
File
Objects
Help
Clear
Exit
Ellipse

Lab 3 - Variant 21
File
Objects
Help
Point
Line
Rectangle
Ellipse

Висновки

Під час виконання лабораторної роботи було закріплено навички роботи з візуальними елементами управління Win32 API (Toolbar, Tooltips), впроваджено динамічне керування пам'яттю для масивів об'єктів та реалізовано специфічні алгоритми побудови геометричних фігур із використанням поліморфізму.

У ході роботи було реалізовано:

- Динамічний масив вказівників `Shape** pcshape` розміром N = 121 для збереження створених фігур (відповідно до індивідуального варіанта 21)

- Динамічне введення об'єктів за допомогою миші в режимі «гумового сліду» із застосуванням штрихової лінії (`PS_DASH`) та XOR-режиму малювання (`R2_NOTXORPEN`)

- Механізм поліморфного виведення графічних примітивів усередині обробника `WM_PAINT` за допомогою віртуального методу `Draw()`, включаючи індивідуальну логіку малювання прямокутника (з центру) та еліпса (із заповненням).

- Створення панелі інструментів (Toolbar) за допомогою класу `TOOLBARCLASSNAMEW` для швидкого вибору інструментів малювання, що дублюють пункти мен

- Реалізація підказок (Tooltips) для кнопок панелі інструментів за допомогою обробки сповіщення `TTN_GETDISPINFOW`

- Динамічне маркування активного інструмента в меню та на панелі інструментів за допомогою повідомлення `WM_INITMENUPOPUP` та `TB_CHECKBUTTON`